



Модуль 0. Пролог: зачем это нужно

0.1. Синхронный мир и его предел

Что значит «синхронный»?

Представьте кассира в банке. Клиент подходит к окошку, кассир принимает его документы, идёт в архив, ищет нужную папку, возвращается, оформляет операцию. Пока кассир ходит в архив, все остальные клиенты стоят и ждут. Кассир не может переключиться на следующего — он занят.

Это синхронная модель. В программировании она означает: программа выполняет одну инструкцию за другой, и если встречается операцию, которая требует ожидания (например, чтение из базы данных или ответ от удалённого сервера), весь поток исполнения замирает и ждёт.

В веб-разработке классический пример — сервер на Flask или Django (в синхронном режиме):

```
In [ ]: @app.get("/data")
def get_data():
    result = database.query("SELECT * FROM users") # <- здесь сервер ждёт
    return {"users": result}
```

Пока база данных выполняет запрос и возвращает результат, поток, обрабатывающий этот HTTP-запрос, ничего не делает. Он заблокирован.

Поток (thread) — что это?

Поток — это последовательность инструкций, которые процессор выполняет. Можно представить поток как «дорожку» в мозге программы. В один момент времени одно ядро CPU может выполнять инструкции только одного потока.

Когда поток встречается операцию ввода-вывода (I/O — Input/Output), например:

- чтение с диска,

- отправку запроса в сеть,
- ожидание ответа от базы данных,

процессор не может ускорить это ожидание. Данные ещё не пришли. И поток просто стоит.

Парадокс: 10 000 пользователей $\neq$ 10 000 ядер

Современный сервер может иметь 8, 16, 32 ядра CPU. Но одновременно к нему могут обращаться тысячи пользователей. Если под каждого пользователя выделять отдельный поток, получается проблема:

- Каждый поток потребляет память (стек вызовов, регистры, служебные структуры ОС). Обычно это от 1 до 8 МБ на поток.
- 10 000 потоков = 10-80 ГБ оперативной памяти только на стеки.
- Операционная система тратит время на переключение между потоками (context switch). При тысячах потоков она тратит больше времени на переключение, чем на полезную работу.

Это называется **C10K Problem** (проблема 10 000 одновременных соединений). Она была сформулирована в конце 1990-х, когда веб начал масштабироваться, и классические серверы на одном потоке на соединение перестали справляться.

Что такое I/O-bound и CPU-bound?

Не все задачи одинаковы. Разделим их на два типа:

CPU-bound (ограничены процессором) — задачи, где процессор постоянно занят вычислениями. Примеры:

- перемножение матриц,
- обучение нейросети,
- обработка изображения.

Если дать такой задаче больше ядер — она ускорится.

I/O-bound (ограничены вводом-выводом) — задачи, где процессор большую часть времени ждёт. Примеры:

- обработка HTTP-запроса, который читает данные из PostgreSQL,

- запрос к внешнему API,
- чтение файла с диска.

В веб-приложениях подавляющее большинство операций — I/O-bound. Сервер ждёт базу данных, ждёт кэш, ждёт ответ от ML-модели. В это время CPU простаивает.

Почему ML-сервисы особенно чувствительны к задержкам?

ML-сервис — это не просто «верни JSON». Он часто включает цепочку операций:

1. Принять запрос (HTTP).
2. Валидировать входные данные (Pydantic).
3. Загрузить фичи из базы или кэша.
4. Вызвать модель для inference.
5. Сохранить результат в лог.
6. Вернуть ответ.

Шаги 3 и 5 — чистый I/O. Шаг 4 — CPU-bound (или GPU-bound). Если сервер синхронный, то во время шага 4 весь поток заблокирован. Но ведь во время шага 3 (ожидание базы) CPU тоже ничего не делает!

Идея: пока один запрос ждёт ответа от базы, сервер мог бы обработать другой запрос. Это и есть суть асинхронности — не ждать зря.

0.2. Ландшафт решений

Три подхода к масштабированию

Есть три основных способа обслужить множество клиентов одновременно:

1. Многопоточность (threading)

Создавать под каждый запрос новый поток. Python имеет модуль `threading`, но с ним есть фундаментальная проблема — **GIL (Global Interpreter Lock)**.

GIL — это мьютекс (блокировка) внутри интерпретатора CPython, который позволяет только одному потоку выполнять Python-код в один момент времени. Даже на 16-ядерном процессоре два потока Python не могут параллельно выполнять байт-код.

Зачем тогда GIL? Он упрощает управление памятью (garbage collection) и защищает внутренние структуры интерпретатора от состояний гонки (race conditions). Без GIL Python был бы медленнее в однопоточном режиме и сложнее в реализации.

Вывод: многопоточность в Python хороша только для I/O-bound задач, где потоки ждут и передают управление друг другу. Для CPU-bound — бесполезна.

2. Мультипроцессинг (multiprocessing)

Запускать несколько процессов Python вместо потоков. Каждый процесс имеет свой интерпретатор, свою память, свой GIL. Процессы действительно работают параллельно на разных ядрах CPU.

Но:

- Создание процесса дорого (сотни миллисекунд).
- Процессы не разделяют память напрямую — нужна сериализация (pickle) для обмена данными.
- Для I/O-bound задач это избыточно: ты создаёшь тяжёлый процесс, чтобы тот просто ждал ответа от сети.

Вывод: мультипроцессинг — решение для CPU-bound задач (например, запуск inference модели в отдельном процессе), но не для обработки тысяч I/O-операций.

3. Асинхронность (asyncio)

Вместо создания тысяч потоков или процессов используется **один поток** и **цикл событий (event loop)**. Программа явно указывает: «здесь я буду ждать ответа от базы, пока жду — займись другими задачами». Когда ответ придёт — вернись ко мне.

Это **кооперативная многозадачность**: задачи сами уступают управление, а не вытесняются операционной системой.

Преимущества:

- Один поток, тысячи корутин (лёгких «задач» внутри event loop). Корутина занимает килобайты памяти, а не мегабайты.
- Нет накладных расходов на переключение контекста ОС.
- Идеально для I/O-bound нагрузки.

Недостатки:

- CPU-bound задачи блокируют весь цикл событий. Если внутри корутины выполнить тяжёлое вычисление — все остальные запросы встанут.
- Код сложнее: нужно явно помечать асинхронные точки (`async / await`).

Почему FastAPI, а не Flask/Django/Express/Go?

Flask — синхронный фреймворк (хотя есть асинхронные обёртки, они вторичны). Он создавался в эпоху, когда веб был проще. Для I/O-bound нагрузки приходится крутить множество процессов-воркеров (Gunicorn), что неэффективно.

Django — мощный, но монолитный. Django 3.1+ поддерживает ASGI, но его экосистема исторически синхронна. ORM Django до сих пор не полностью асинхронна.

Express (Node.js) — асинхронный по дизайну, но на JavaScript. Node.js использует тот же event loop (через libuv). Однако:

- Python доминирует в ML-экосистеме (PyTorch, TensorFlow, scikit-learn, Hugging Face).
- Переписывать ML-логику на JavaScript невозможно или неразумно.

Go — отличный язык для бэкенда, с горутинами (лёгкими потоками в рантайме Go). Но:

- В Go нет экосистемы для ML.
- Интеграция Python-моделей с Go требует RPC/HTTP-прослойки, что добавляет сложности.

FastAPI строится на трёх столпах:

1. **Starlette** — лёгкий ASGI-фреймворк (асинхронный по определению).
2. **Pydantic** — валидация данных через типы (с ядром на Rust в V2).
3. **Python** — полный доступ к ML-экосистеме.

FastAPI позволяет писать асинхронный код, который одновременно:

- обрабатывает тысячи I/O-операций (чтение из БД, кэша, внешних API),
- вызывает Python-библиотеки для ML (без необходимости переписывать их),
- валидирует данные со скоростью, близкой к компилируемым языкам (благодаря Pydantic V2).

Roadmap курса: от «Hello, World» до продакшн-архитектуры

Этот курс построен по принципу **от фундамента к архитектуре**. Мы не начнём сразу с FastAPI, потому что без понимания того, как работает `async / await` и event loop, FastAPI останется магией.

Последовательность:

1. **Модуль 1** — углубимся в Python: итераторы, генераторы, контекстные менеджеры, типизация. Это инструменты, которые лежат в основе асинхронности.
2. **Модуль 2** — изучим `asyncio`: event loop, корутины, задачи, синхронизацию. Поймём, как Python управляет конкурентностью.
3. **Модуль 3** — разберём сетевые протоколы (TCP, HTTP, WebSocket), чтобы понимать, что именно мы оптимизируем.
4. **Модуль 4** — перейдём к FastAPI: ASGI, маршрутизация, Pydantic V2, обработка ошибок, middleware, lifespan.
5. **Модуль 5** — Dependency Injection: как строить масштабируемую архитектуру.
6. **Модуль 6** — асинхронные базы данных: SQLAlchemy 2.0, Alembic, Redis.
7. **Модуль 7** — безопасность: аутентификация, авторизация, криптография.
8. **Модуль 8** — тестирование асинхронного кода.
9. **Модуль 9** — интеграция ML: inference, фоновые задачи, streaming.
10. **Модуль 10** — продакшн: Docker, мониторинг, масштабирование.

11. **Модуль 11** — архитектурные паттерны: чистая архитектура, REST vs Event-Driven, микросервисы.
12. **Модуль 12** — кейсы и финальный проект.

Ключевые вопросы, на которые вы сможете ответить после курса

- Почему `time.sleep(10)` внутри асинхронного endpoint убивает производительность всего сервера?
- Как загрузить ML-модель один раз при старте и использовать её в тысячах запросов?
- В чём разница между `BaseHTTPMiddleware` и чистой ASGI-middleware, и почему это важно для latency?
- Как организовать фоновую обработку, чтобы не потерять задачу при перезапуске сервера?
- Почему REST не всегда лучший выбор для ML-сервисов, и когда стоит перейти на Event-Driven архитектуру?

Итог модуля 0

Концепция	Суть
Синхронность	Выполняем по порядку, ждём завершения каждой операции.
I/O-bound	Задачи, где процессор ждёт (сеть, диск, БД).
CPU-bound	Задачи, где процессор считает (ML, математика).
GIL	Блокировка в CPython, ограничивающая параллелизм потоков.
Многопоточность	Много потоков, но GIL мешает для CPU-bound; приемлемо для I/O.
Мультипроцессинг	Много процессов, параллелизм настоящий, но дорого.
Асинхронность	Один поток, явная передача управления, идеально для I/O.
FastAPI	ASGI + Pydantic + Python = асинхронный бэкэнд с доступом к ML.

В следующем модуле мы начнём с фундамента Python — итераторов и генераторов — потому что именно они легли в основу механизма `async / await`.